

Offline-First

Lecture 10: local-first data, sync and conflict resolution

Who's teaching this course

Dinu-Ștefan Rusu

dinu_stefan.rusu@upb.ro

Microsoft Teams course channel

rusudinu.com

45+

Flutter apps shipped

400k

installs across them

Appeared in, spoken at & partnered with



TODAY

What you should leave with



Design for offline

Why offline is a normal state, and why the UI reads local data, never the server



Build the local layer

A Drift schema with sync metadata, a reactive watch() query, and an outbox table



Write the sync engine

Push loop, delta pull, checkpoints, and the background triggers you can actually rely on



Resolve conflicts honestly

Clock skew, hybrid logical clocks, merge by field, and what a CRDT guarantees

WHY OFFLINE-FIRST

The network is not a dependency

It is a background utility that is often, briefly, absent

Offline is a state, not an error

< 1 ms

A local read

an indexed SQLite query on the device: no radio, no server, no failure mode

40–200 ms

A good round trip

4G or 5G, warm connection, server in the same region. On a bad link: seconds

30 s

Your timeout

how long the user waits before the request is abandoned

Lifts, tunnels, basements, planes, trains, festivals, hospital wards, rural roads, and the hotel Wi-Fi that resolves DNS and then routes nothing. All of them are ordinary conditions for your users.

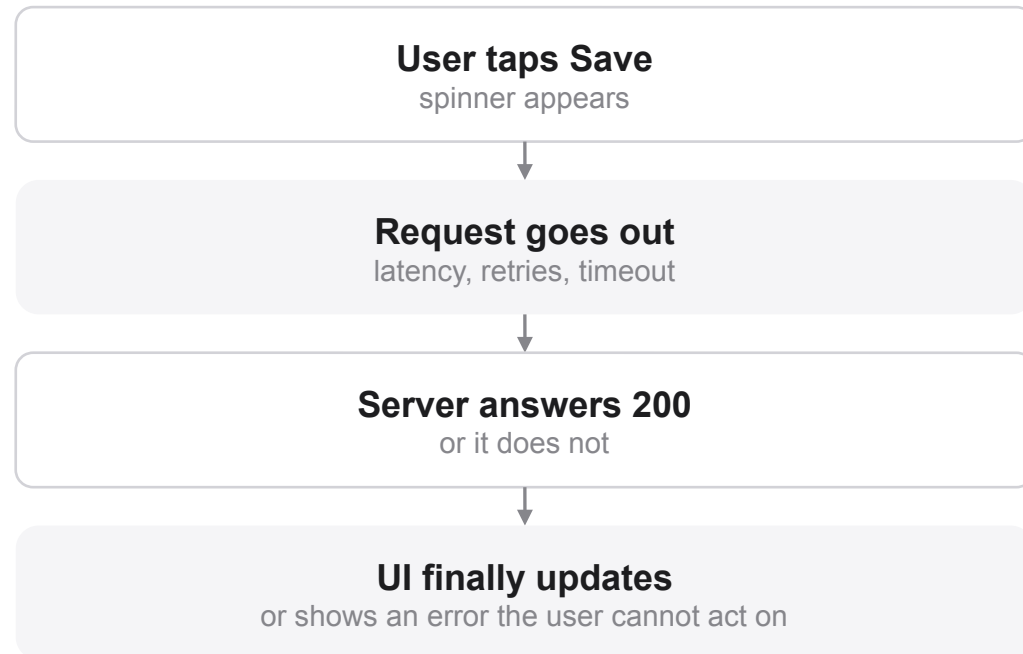
Worse than no signal is “lie-fi”: the interface is up, the TCP connection opens, and the request hangs until your timeout fires. An app that treats the network as available-or-not gets this wrong every time.

An offline-first app has no offline error path, because being offline is not an error.

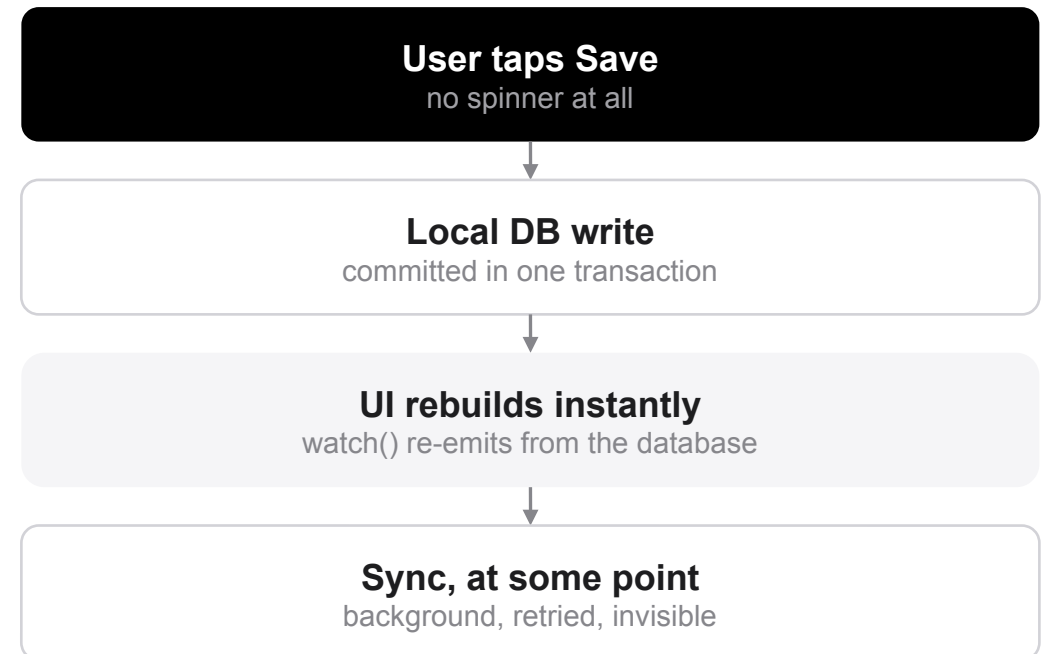
The inversion of truth

The same user action, in the two architectures. Only one of them still works in a lift.

Online-first: $UI = f(\text{network response})$



Offline-first: $UI = f(\text{local database})$



The network stops being on the critical path of a user action. It becomes a background job that reconciles two copies of the truth, and the user never waits for it.

CAP, on a device in your pocket

CAP is a statement about what happens during a partition: when the two sides cannot talk, you either refuse to answer, or you answer with data that may be stale. You cannot have both.



Partition tolerance

Not a choice on mobile. The partition is not a rare fault you engineer against. It is the resting state of a phone in a lift.



Availability

You keep it. Every read is answered from the local database and every write is accepted, signal or no signal.



Consistency

You give it up. What you get instead is eventual consistency, and the cost of it is the whole second half of this lecture.

The network picks P for you. The only thing you choose is whether the app stays useful while partitioned, or freezes waiting for data it cannot reach.

THE LOCAL DATABASE

One source of truth, on the device

Schema, reactive reads, and an outbox for every write

Firestore's cache, or your own SQLite

	Firestore offline cache	Drift (SQLite)
Types	dynamic maps, checked at runtime	compile-time checked SQL and rows
Reactive UI	snapshots() streams	watch() streams, on any query you can write
Offline queries	no local indexes: full scans, and jank	your indexes work offline too
Transactions	online only, no offline atomicity	full local ACID transactions
Cache control	opaque LRU; you cannot pin a document	it is your file; nothing is evicted
Cost	billed per document read, sync catch-up included	flat: whatever your backend costs

Firestore is a managed sync engine you cannot open. For a project that must query, transact and search offline, own the database, or take a middle path like PowerSync. → Lecture 5, the backend it syncs with

Every synced table carries sync metadata

```
// lib/db/tables.dart

class Notes extends Table {
  // client-generated UUID: two offline devices must never
  // invent the same primary key
  TextColumn get id => text();
  TextColumn get title => text().withDefault(const Constant(''))();
  TextColumn get body => text().withDefault(const Constant(''))();

  // ---- the four columns that make a table syncable ----
  DateTimeColumn get updatedAt => dateTime();
  DateTimeColumn get deletedAt => dateTime().nullable();
  BoolColumn get pendingSync =>
    boolean().withDefault(const Constant(true))();
  IntColumn get serverVersion =>
    integer().withDefault(const Constant(0))();

  @override
  Set<Column> get primaryKey => {id};
}
```

id: a UUID, made on the client

Auto-increment collides: two offline devices both create row 42, and one of them loses.

updatedAt: when this device last edited

The input to any timestamp-based resolution. Its weakness is slide 21.

deletedAt: the tombstone

A delete is a write. Never issue a local DELETE for a synced row.

pendingSync: the isDirty flag

True from the moment you touch the row until the server has acknowledged it.

serverVersion: what we last saw

Lets the resolver tell “we are ahead” from “both sides moved”.

The UI reads local data

```
@DriftAccessor(tables: [Notes])
class NotesDao extends DatabaseAccessor<AppDb> with _$NotesDaoMixin {
    NotesDao(super.db);

    // A live query. Drift re-runs it and re-emits whenever the
    // notes table changes, no matter who changed it: the user,
    // or the sync engine writing rows it pulled from the server.
    Stream<List<Note>> watchActive() =>
        (select(notes)
            ..where((n) => n.deletedAt.isNull())
            ..orderBy([(n) => OrderingTerm.desc(n.updatedAt)]))
            .watch());
}
```

```
StreamBuilder<List<Note>>(<
    stream: dao.watchActive(),
    builder: (_, snap) => NoteList(snap.data ?? const []),
)
```

One stream, two writers. The user edits a note and the list updates; the sync engine pulls a change from the server and the same list updates. The widget never knows which happened.

There is no loading state for the network here, only the first-launch case where the database is genuinely empty.

Filtering `deletedAt IS NULL` in the query is what makes tombstones invisible to the UI while they stay visible to the sync engine.

Drift's `watch()` is a `Stream`, so everything you learned about `StreamBuilder` and `Riverpod` still applies. → [Lecture 4](#)

Writes go to the table and an outbox

```
// One transaction. Either both rows land, or neither does.
Future<void> saveNote(Note n) => db.transaction() async {
    final row = n.copyWith(
        updatedAt: DateTime.now(), pendingSync: true);
    await into(notes).insertOnConflictUpdate(row);
    await into(outbox).insert(OutboxCompanion.insert(
        id: uuid.v4(), entity: 'note', entityId: n.id,
        op: 'upsert', payload: jsonEncode(row.toJson()),
        queuedAt: DateTime.now(),
    ));
});

// A delete is a write. Never a DELETE statement.
Future<void> deleteNote(String id) => db.transaction() async {
    await (update(notes)..where((n) => n.id.equals(id))).write(
        NotesCompanion(deletedAt: Value(DateTime.now()),
            pendingSync: const Value(true)));
    await into(outbox).insert(/* op: 'delete', entityId: id */);
};
```

Why not just call the API?

Because the call can fail, and then your local row and the server disagree with nothing recording that fact.

The outbox is that record: a durable, ordered list of intents that survives the process being killed mid-request.

One transaction, two effects.

The data row and the intent to sync it commit together, so the queue can never drift out of step with the data.

The UI already updated: `watch()` fired the moment the transaction committed, long before any packet left the device.

THE SYNC ENGINE

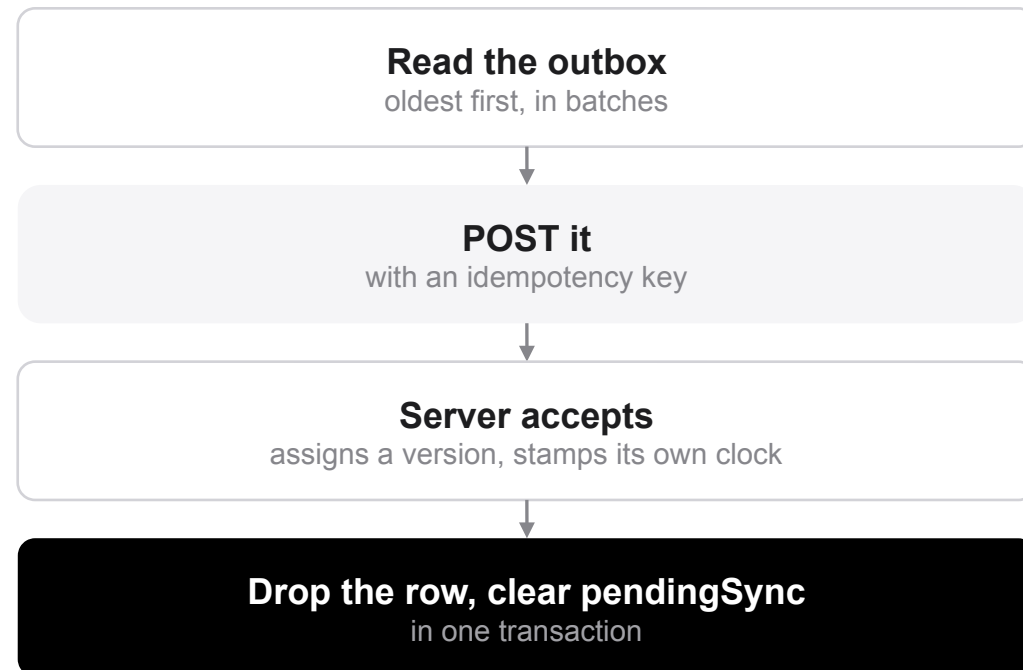
Two loops and a checkpoint

Push what we changed, pull what they changed, remember where we got to

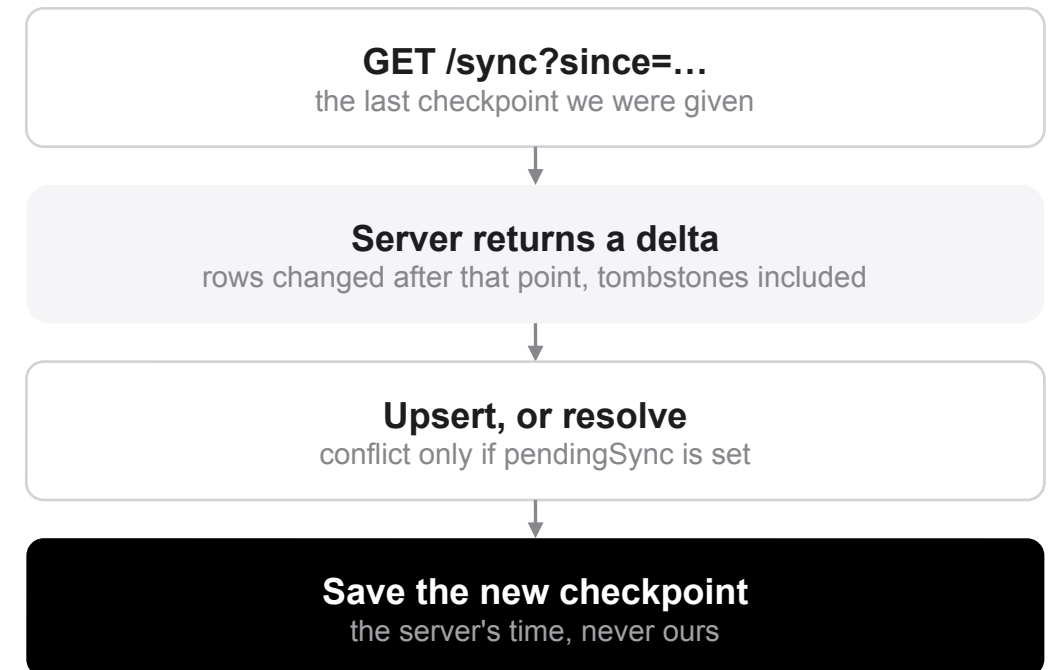
The two loops

Nothing here runs while the user is waiting. Both loops are triggered by events, and both are safe to interrupt.

Push: what this device changed



Pull: what everyone else changed



The checkpoint is the whole protocol. Store the timestamp the server reported, not the one your device read. Otherwise clock skew makes you skip rows you never downloaded.

Draining the outbox

```
Future<void> drainOutbox() async {
    final batch = await (select(outbox)
        ..orderBy([(o) => OrderingTerm.asc(o.queuedAt)])
        ..limit(50)).get();

    for (final item in batch) {
        try {
            await api.push(item, idempotencyKey: item.id);
            await db.transaction() async {
                await (delete(outbox)..where((o) => o.id.equals(item.id))).go();
                await (update(notes)
                    ..where((n) => n.id.equals(item.entityId)))
                    .write(const NotesCompanion(pendingSync: Value(false)));
            });
        } on ApiFailure catch (e) {
            if (e.permanent) { await _park(item); continue; } // 4xx
            break; // transient: stop, keep the order, try again later
        }
    }
}
```

FIFO, and stop on the first transient failure.

Skipping ahead reorders the user's edits, and the server sees an update for a row it has not been told about yet.

The idempotency key is the outbox row id.

A reply lost on the way back means you push the same intent twice. The server must recognize it and do nothing. → Lecture 5

Park poison messages.

A permanent 4xx will fail identically forever. Move it aside, surface it, and let the rest of the queue through.

Delta sync: only what changed

```
Future<void> pull() async {
  var cursor = await meta.checkpoint(); // the server's clock
  while (true) {
    final page = await api.get('/sync', {'since': cursor});
    await db.transaction(() async {
      for (final r in page.rows) {
        final local = await notesDao.findById(r.id);
        if (local != null && local.pendingSync) {
          await notesDao.put(resolve(local, r)); // slide 24
        } else {
          await notesDao.put(fromServer(r)); // incl. deletedAt
        }
      }
      await meta.setCheckpoint(page.serverCursor);
    });
    if (!page.hasMore) break;
    cursor = page.serverCursor;
  }
}
```

```
GET /sync?since=2026-08-01T09:14:22Z
```

```
{ "serverCursor": "...:31Z",
  "hasMore": false,
  "rows": [
    {"id":"a1","title":"Milk",
     "deletedAt":null,"version":7},
    {"id":"b2","deletedAt":"..."}
  ]
}
```

Deleted rows come down as rows. A row that is simply absent means “not changed”, never “gone”.

Page it. A device offline for a fortnight will not get its backlog in one response.

Whole transaction per page, so an interrupted sync leaves a consistent database and a usable checkpoint.

When to wake the radio

	How it works	Latency	Radio cost	Use it for
Push	server holds a socket open, or sends a data push	real time	high: keep-alives, constant wake-ups	chat, live tracking, shared editing
Poll	a timer fires a request whether or not anything changed	half the interval	high and wasted: most calls return nothing	legacy backends with no push
Delta on trigger	sync when something happened: resume, reconnect, pull-to-refresh	seconds	low: one exchange per real event	almost every app, including yours
Nudge + delta	a silent push says “there is news”, the client then pulls a delta	near real time	low: the payload rides an existing channel	feeds, inboxes, shared lists

Sync on events, not on a timer. App resumed, connectivity returned, user pulled down, or the server said something changed. A fixed interval spends battery on requests that return nothing. → Lecture 8, WebSockets for the push channel · → Lecture 11, what a radio wake-up actually costs

Background sync, and what it gives you

Android: WorkManager.

Periodic work with a 15-minute floor, constraints you can declare (network connected, charging, battery not low), and survival across reboots. Doze still batches it, so 15 minutes means “no sooner than”.

iOS: BGTaskScheduler.

You register an identifier and submit a request. The system decides, based on how often the user opens your app, whether you run at all. Execution is never guaranteed and the window is short.

So do not depend on it.

Treat background sync as an optimization. The syncs you can rely on are the ones you trigger yourself, in the foreground.

```
// the triggers that actually fire
void didChangeAppLifecycleState(AppLifecycleState s) {
    if (s == AppLifecycleState.resumed) sync.now();
}

Connectivity().onConnectivityChanged.listen((r) {
    if (!r.contains(ConnectivityResult.none)) sync.now();
});

// best effort, on top of those
Workmanager().registerPeriodicTask(
    'sync', 'sync',
    frequency: const Duration(minutes: 15),
    constraints: Constraints(
        networkType: NetworkType.connected),
    );
```

Foreground triggers are the contract; background work is a bonus. → Lecture 11: every wake-up costs a radio ramp-up you pay for in battery.

CONFLICTS

Two devices editing the same row

Clocks, resolution strategies, and data types that cannot conflict

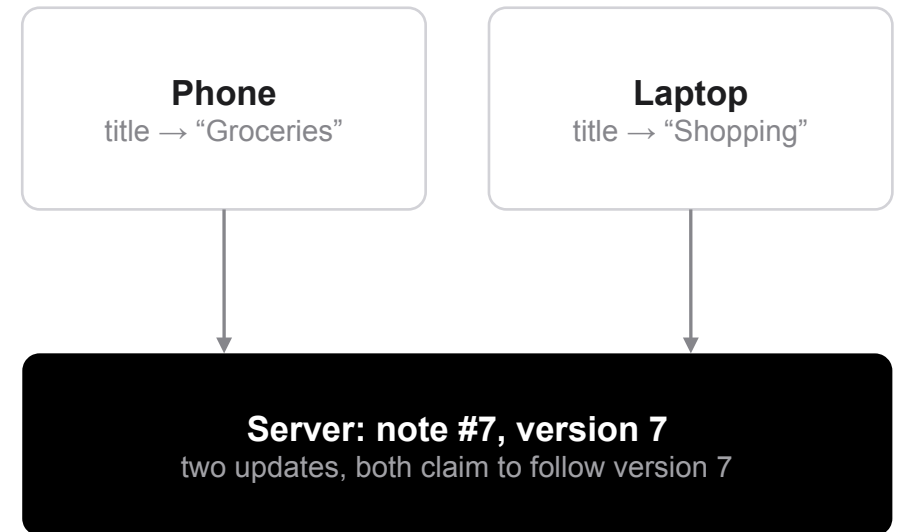
Where conflicts actually come from

A conflict is not a bug and not a race in your code. It is two edits to the same row that were made without either side being able to see the other.

Offline-first guarantees they will happen: the whole design is “accept the write now, reconcile later”. A single user with a phone and a laptop is enough.

The server sees both updates arrive claiming to modify version 7. It has to pick, merge, or reject, and each of those affects someone's work.

You cannot prevent conflicts. You can only decide, in advance and in writing, what happens when one occurs.



Both edits were made while offline. Neither device saw the other. Nothing has been overwritten yet. That decision is still yours to make.

Design the resolution rule before you need it. Otherwise the rule is whatever your ORM happened to do.

CONFLICTS

Device clocks cannot order events

Every device believes its own clock. NTP drift, a user setting the time by hand, a flat battery, a cheap RTC: a few seconds of disagreement is normal, and minutes is not rare.



The later write carries the earlier timestamp. Last-write-wins compares 09:14:07 with 09:13:55 and keeps "Milk". The user's newest edit is discarded with no error, no log and no undo. That happens whenever two clocks disagree by more than the gap between two edits.

Logical clocks: order without a clock

```
// Hybrid Logical Clock: readable time, trustworthy order.
class Hlc implements Comparable<Hlc> {
    final int wallMs;    // milliseconds since epoch
    final int counter;   // ties inside the same millisecond
    final String node;   // replica id, deterministic tiebreak

    Hlc tick(int nowMs) => nowMs > wallMs    // a local event
        ? Hlc(nowMs, 0, node)
        : Hlc(wallMs, counter + 1, node);

    // a timestamp arrived from another replica
    Hlc receive(Hlc r, int nowMs) {
        final w = [nowMs, wallMs, r.wallMs].reduce(max);
        if (w == wallMs && w == r.wallMs)
            return Hlc(w, max(counter, r.counter) + 1, node);
        if (w == wallMs)    return Hlc(w, counter + 1, node);
        if (w == r.wallMs) return Hlc(w, r.counter + 1, node);
        return Hlc(w, 0, node);
    }
}
```

Lamport clock: a counter, not a time.

Local event: $L = L + 1$. Send: attach L . Receive: $L = \max(L, L_{\text{received}}) + 1$.

Guarantee: if a caused b , then $L(a)$ is less than $L(b)$. The converse is false: a smaller counter does not prove causality, and may just mean the two events were concurrent.

Vector clock: one counter per replica.

Can actually detect concurrency, at the cost of state that grows with the number of devices.

HLC: the practical compromise.

Ordering that never contradicts causality, timestamps that still read like real times, 64 bits on the wire. Use it as your `updatedAt`.

Four strategies, honestly ranked

	How it decides	What it costs you	Reach for it when
Last-write-wins	the whole row with the highest timestamp wins	silent data loss: the loser's edit is gone and nobody is told	one user, one device at a time, low-value fields
Server-authoritative	the server recomputes and its answer is final	an accepted offline edit can be rejected later; you must show the user	money, stock, bookings, anything with an invariant
Merge by field	compare column by column; only changed fields overwrite	one resolver per entity, written and tested by you	documents and forms with independent fields
CRDT	merge() is defined so every order of arrival converges	metadata: tags, tombstones, version vectors, and their growth	counters, sets, and collaborative text

Most apps need two of these, not one. Merge by field for the user's own documents, server-authoritative for anything that must add up. Reach for a CRDT when the data type genuinely fits one.

A resolver you can actually ship

```

/// Merge by field. Needs a per-field timestamp: the row stores
/// titleAt / bodyAt as well as updatedAt.
Note resolve(Note local, ServerNote remote) {
    // nothing pending locally: the server is simply newer
    if (!local.pendingSync) return fromServer(remote);
    // we already have everything they have: keep ours, push it
    if (remote.version <= local.serverVersion) return local;

    // real conflict: both sides moved past version N
    return local.copyWith(
        title: _pick(local.title, local.titleAt,
                     remote.title, remote.titleAt),
        body: _pick(local.body, local.bodyAt,
                   remote.body, remote.bodyAt),
        deletedAt: local.deletedAt ?? remote.deletedAt, // delete wins
        serverVersion: remote.version,
        pendingSync: true, // a new value, push it back
    );
}

```

```

T _pick<T>(T mine, Hlc mineAt,
          T theirs, Hlc theirsAt) =>
    mineAt.compareTo(theirsAt) >= 0
        ? mine : theirs;

```

Per field it is still last-write-wins, but only one column is affected instead of the whole record. A concurrent title and body edit now both survive.

`_pick` compares HLCs, so it is deterministic: both devices reach the same answer without talking to each other.

Delete beats edit here. That is a product decision, not a technical one. Write it down, and consider an undo instead.

Push the merged row back.

If the resolver only fixes the local copy, the two sides never converge.

CRDTs: convergence without coordination

A CRDT is a data type whose merge function is defined so that replicas which have received the same set of updates end up in the same state, with no leader, no locks and no round trip. That property is called strong eventual consistency.



Commutative

$\text{merge}(a, b)$ equals $\text{merge}(b, a)$. Messages may arrive in any order, from any device, over any path.



Associative

Grouping does not matter, so replicas can gossip in whatever topology the network gives them.



Idempotent

$\text{merge}(a, a)$ equals a . Receiving the same update twice is a no-op, which is what makes retries free.

Convergent is not the same as correct. A CRDT guarantees every replica agrees on the answer. It does not guarantee the answer is what the user meant, and it cannot enforce “the balance must never go below zero”, because that needs coordination by definition.

A counter that survives being merged

A plain integer cannot merge. Phone says 4, laptop says 3: is the answer 4, or 7? You cannot tell without the history. So keep the history: one slot per replica, each replica only ever increments its own slot.

	Phone's slot	Laptop's slot	Tablet's slot	Value = sum
Phone's state	3	1	0	4
Laptop's state	1	2	0	3
Merge = max per slot	3	2	0	5
Tablet then adds 1	3	2	1	6

The merge is a maximum, not an addition, which is why receiving the same state twice changes nothing. The value is the sum of the slots.

This is a G-Counter: grow-only. To support decrements you keep two of them, one for increments and one for decrements, and subtract the totals: that is a PN-Counter. A maximum cannot un-count.

Deleted is a row, not a missing row

Delete a row locally and the next pull cannot tell “this was deleted here” from “this has not reached me yet”. The server sends it back, and the item the user deleted reappears.

So a delete is a write: set `deletedAt`, keep the row, filter it out of every query the UI runs. The tombstone is what propagates the deletion.

The same problem inside a set.

An OR-Set gives every add a unique tag. A remove takes away only the tags that replica has observed. A concurrent add survives: the remover never saw its tag, so it could not have meant to remove it.

Tombstones are not free.

They accumulate forever unless you collect them. Either prove every replica has seen the deletion, or set a retention window: purge tombstones older than, say, 90 days, and make any device that has been offline longer do a full resync instead of a delta.

Add wins, because the remover could not have observed it. That is a defensible default for a shopping list, and the wrong one for a bank ledger.

OR-Set, one worked case

A: add “milk”

```
adds = {(milk, a1)}
```

B: sees a1, removes “milk”

```
removed = {a1}
```

A, concurrently: add “milk” again

```
adds = {(milk, a1), (milk, a2)}
```

merge: adds minus removed

```
= {(milk, a2)}, milk is present
```

Before next week



Recap

Offline is a normal state; the local database is what the UI reads, always

One transaction writes the row and the outbox intent together

Delta sync on a server-issued checkpoint, never on your device's clock

Wall clocks cannot order events; HLCs can, and still read like times

Deleted is a row. LWW is simple and lossy. CRDTs converge, but convergence is not correctness



This week: on your project

Add Drift, with id (UUID), updatedAt, deletedAt and pendingSync on every synced table

Replace one FutureBuilder with a watch() stream

Add the outbox table and drain it on resume and on reconnect

Then turn on airplane mode and use your own app for five minutes



Read more

drift.simonbinder.eu: schema, DAOs, migrations

crdt.tech: the CRDT catalog and the original papers

Kleppmann, Designing Data-Intensive Applications, ch. 5

inkandswitch.com/local-first

pub.dev/packages/workmanager · powersync.com